LiveRecall

Revision-aware live captions with bounded recovery and per-client delivery

Ng De Heng • Foo Juinn Kai • Jay Kim • Ong Cheng Leong

1 System architecture

The supplied deterministic source emits provisional revisions and a final caption for each segment. A WebSocket server validates and distributes that stream to many browser clients. The browser replaces revisions in place, retains recent finals, and provides a local 15/30/60-second recall view.

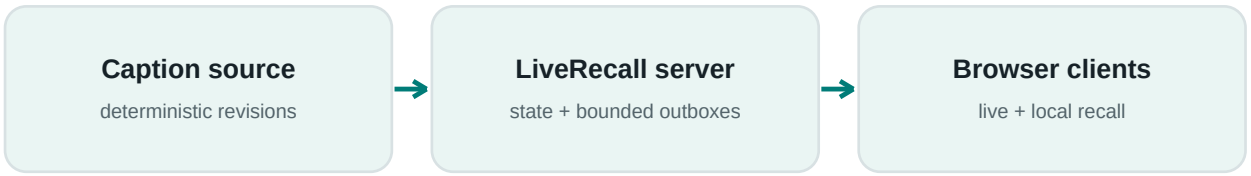

Figure 1. Required data path. The server keeps bounded session history and separate per-client outboxes.

Protocol

- The source sends **SOURCE_HELLO**, then **CAPTION_UPDATE** objects with session, segment, revision, source timestamp, final flag, and text.
- A student sends **CLIENT_HELLO** with the latest finalized segment/revision and a requested history window. The server replies with oldest-first **HISTORY**, then live updates.
- The browser stores one current caption per segment, distinguishes provisional and final text, reconnects automatically, and requests only recent missed finals after a gap.

The same submitted server and browser are used for the deterministic application demo and the measured tests; speech recognition is outside the mandatory path.

2 Correctness and recovery

The server accepts a revision only when it is newer than the stored revision and the segment has not already become final. This prevents stale updates from overwriting completed text. Final captions enter a bounded recent-history deque. On reconnect, the client sends its latest finalized cursor; the server returns only newer finals within the 60-second recovery limit. The browser deduplicates history against local segment state.

A late joiner starts with cursor zero and receives recent context before live updates. Snapshotting history and subscribing a client occur without an intervening await, avoiding a gap or duplicate around the handoff. The browser holds at least 60 seconds of recent finalized captions locally for ordinary recall.

3 Semantic backpressure

Each client has a separate bounded outbox and sender task, so one slow receiver cannot block the source handler or normal readers. Under normal conditions the full revision stream is forwarded. When the network path falls behind, TCP acknowledgement feedback holds provisional revisions while finals proceed. Application-level ping/pong feedback detects a reader that has stopped consuming even when TCP has accepted bytes; that reader is held and then paced near its measured reading rate.

- Unsent obsolete provisional revisions can be replaced with the latest state of their segment; finalized captions remain durable.
- Several pending finals may be bundled in one catch-up HISTORY frame, reducing the number of reads a slow client must perform.
- Per-client pending work is capped; a client that falls too far behind is closed so it can resume through bounded history rather than growing an unlimited queue.

Trade-off

Coalescing lowers provisional update coverage during overload but protects final-caption delivery and freshness. The control loops should return to full-update delivery once lag clears. The browser demo separately verifies revision replacement, local recall, reconnect, and late join.

The six server correctness checks (**bash tests/run_checks.sh**) pass on the machine used for every reported run.

4 Public experimental method

The supplied harness, unmodified, starts a fresh server for each run and logs caption receipt by simulated clients. All runs used native Ubuntu 26.04 (Linux 7.0, 16 cores) with server, source and clients on one machine; T5 applies Linux tc/netem to loopback. Each scenario has three independent runs and median metrics. Caption Age is client receive time minus source timestamp. Freshness uses the public 500 ms threshold.

Required public tests

Test	Condition	P50	P95	P99	Fresh	Finals	Compl.	Cover.
T1	20 normal clients	1.2	1.9	2.1	100.0%	100.0%	100.00%	100.00%
T2	100 normal clients	3.3	6.1	6.9	100.0%	100.0%	100.00%	100.00%
T3	100; 10% slow	3.1	6.1	3603	98.4%	97.0%	100.00%	94.39%
T4	100; 20% reconnect	2.9	5.1	6.2	100.0%	98.4%	100.00%	98.25%
T5	100; delay + 3% loss	253	691	960	91.3%	87.8%	100.00%	97.97%

Latencies in ms. Every run reached 100% final completeness, with no connection-error, duplicate or stale events. T3: normal clients stayed 100% fresh; slow readers had P50 181 ms and P95 7.5 s, which raises the overall P99. T4: recovery P95 was 3.4 ms.

T5 network profiles

Profile	P50	P95	P99	Fresh	Finals	Compl.	Cover.
100 +/- 20 ms delay	203	250	267	100.0%	100.0%	100.00%	100.00%
120 +/- 30 ms delay; 3% loss	253	691	960	91.3%	87.8%	100.00%	97.97%
3% loss	3.4	154	217	100.0%	100.0%	100.00%	99.93%

Public network profiles on loopback, 100 clients, 40-second window, three runs each; every run reached 100% completeness. Delay shifts the whole distribution by about one round trip. Loss only hurts the tail: a lost segment is resent after TCP's 200 ms minimum retransmission timeout and every caption behind it waits (head-of-line blocking), so P95/P99 sit near that timeout.

5 Before/after comparison

Before: the team's earlier queue-based server (PR 8), which reacts only when its own outbox backs up. After: the final design. Same machine and harness, three runs each.

Scenario	Compl. before	Compl. after	Metric	Before	After
T3, 10% slow readers	95.13%	100.00%	Slow-reader P50	14.7 s	181 ms
30% slow readers	85.39%	100.00%	Slow-reader P50	14.7 s	204 ms
Slow readers, 300 clients, 2 Mbit/s	95.00%	100.00%	Slow-reader P50	14.6 s	482 ms
700 clients, 2 Mbit/s	89.51%	99.99%	P95	7.0 s	2.0 s
300 clients, 2 Mbit/s	100%	100%	P50	71 ms	370 ms

A slow reader's websockets library and kernel buffers absorb thousands of ~42-byte messages before the earlier server's outbox backs up, so its slow readers ended about 15 s behind and missed finals. The final design detects the pause at the library through ping/pong and paces the reader. The last row is a regression, explained below.

6 Failures and trade-offs

What did not work (each measured, then dropped)

• **Receive-window signal.** A slow reader's advertised TCP window stayed at 65,536 B with about 70 unread messages (128 KB buffers, ~42-byte messages), so TCP cannot reveal a slow reader. Hence the ping/pong loop.

• **TCP_NOTSENT_LOWAT.** Keeping unsent data in the outbox, where it can be replaced, gave nothing on a 2 Mbit/s bottleneck: the backlog was already in flight, not unsent.

• **One ping per 8 messages.** The pause point was known only to within 8 messages, so the server overestimated what was still queued after a stall (12 vs about 8). Readers taking 2 s per message idled about 10 s and missed the last final (99.87%). A ping on every 4th message, with the stall timed from the middle of the uncertainty interval, fixed this (100%; T3 slow-reader P50 from 222 to 181 ms).

• **Thresholds from TCP's retransmission timeout and the caption stream's rate.** On a congested link a late pong includes network delay, so the reading rate implied by a stall is wrong: 909 fast clients were paced at 500 clients, and worst-run completeness fell to 94.3% at 700 clients. Reverted.

Trade-offs we kept

• **Slow readers** get fewer provisionals but current text: T3 coverage 94.4% (queue-only server 95.2%), slow-reader P50 181 ms (14.7 s), completeness 100% (95.13%).

• **Under overload**, provisionals wait in the outbox and are replaced while finals go out: coverage 61% vs 99% at 500 clients on 2 Mbit/s, but completeness 99.99% vs 89.5% at 700 clients.

• **Probe precision costs bandwidth.** Each ping adds a pong and its ACK; at 300 clients on 2 Mbit/s P50 is 370 ms vs 71 ms.

• **A slow reader's first 15-20 s are stale** (8% fresh, then about 100%), because its library buffers 16 messages before any signal reaches the server. Shortening this would mean throttling every new client, which the coverage check penalizes.

• **Lost pongs under heavy overload** still cause false holds (518 fast-client pacing decisions across our network runs).

7 Limitations and future work

Results come from one machine with three runs per condition and a deterministic source; the hidden workload may combine conditions we did not test. Both loops read Linux TCP_INFO: on other platforms the network gate is off and a ping still in flight counts as a stall. Future work: a pause signal that survives congestion (for example, reading-progress

reports from the client), pinging less often once a client has proven fast, and a real lecture speech recognizer behind the same protocol.

Sources: CS3103 Assignment 4 LiveRecall brief (17 Oct 2026); supplied harness and code; experiments/ and tests/ in this repository; earlier controlled slow-client study in docs/slow_client_evaluation.md.